



**UNIVERSIDAD  
SERGIO ARBOLEDA**

**Taller 2 corte 2**

**Lenguajes de programación**

**Yslen Natalia Moreno Gutierrez  
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda  
Escuela de ciencias exactas e ingeniería  
Bogotá D.C., 09 de septiembre de 2026*

## 1. Introducción

El objetivo del taller es diseñar gramáticas libres de contexto (GLC) en ANTLR4 capaces de reconocer y evaluar expresiones aritméticas con suma, resta, multiplicación, división y paréntesis. A partir de una gramática base se construyeron tres variantes, cada una cambiando una sola propiedad, para observar cómo la **asociatividad** y la **precedencia** de los operadores modifican el árbol de sintaxis abstracta (AST) y el resultado de la evaluación.

| Gramática | Precedencia   | Asociatividad | Cambio                |
|-----------|---------------|---------------|-----------------------|
| G1        | * / > + -     | Izquierda     | Gramática base        |
| G2        | * / > + -     | Derecha       | Asociatividad         |
| G3        | + - > * /     | Izquierda     | Precedencia invertida |
| G4        | / > * > - > + | Izquierda     | Un nivel por operador |

Tabla 1: Resumen de las gramáticas desarrolladas

Todas las gramáticas se implementaron en Python con el **patrón Visitor**, que recorre el árbol generado por ANTLR y calcula el resultado de cada expresión.

## 2. Preparación del entorno

Se trabajó en Ubuntu. Se creó la carpeta del proyecto y un entorno virtual de Python, en el que se instaló el runtime de ANTLR con la misma versión del generador (4.13.2), requisito para que el código generado funcione.

```
mkdir -p ~/Escritorio/LenguajesProgramacion/Aritmetica
cd ~/Escritorio/LenguajesProgramacion/Aritmetica
python3 -m venv venv
source venv/bin/activate
pip install antlr4-python3-runtime==4.13.2
```

```
santiago-patino@Vivobook:~/Escritorio/asociatividad-precedencia$ echo "Verificar instalaciones previas"
Verificar instalaciones previas
santiago-patino@Vivobook:~/Escritorio/asociatividad-precedencia$ java -version

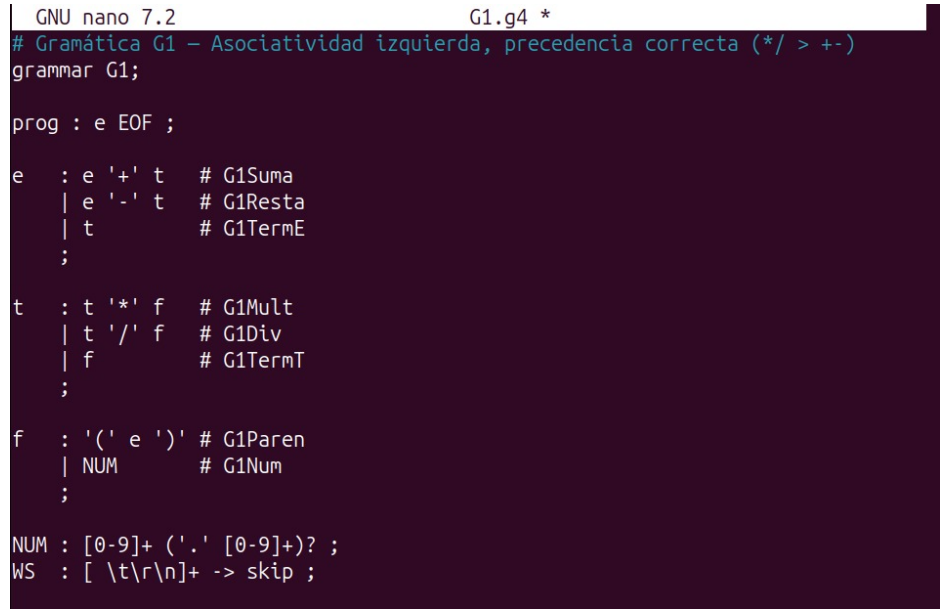
type antlr4
type grun
python3 --version
python3 -c "import antlr4; print(antlr4.__version__)"
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
antlr4 es un alias de `java -jar /home/santiago-patino/antlr-4.13.2-complete.jar`
grun es un alias de `java org.antlr.v4.gui.TestRig`
Python 3.12.3
Traceback (most recent call last):
  File "<string>", line 1, in <module>
AttributeError: module 'antlr4' has no attribute '__version__'
santiago-patino@Vivobook:~/Escritorio/asociatividad-precedencia$
```

Figura 1: Instalaciones previas y verificación de versiones

### 3. Gramática G1: asociatividad izquierda

G1 es la gramática base. Usa tres niveles: **e** (suma y resta), **t** (multiplicación y división) y **f** (paréntesis y números). El nivel más profundo del árbol tiene mayor prioridad, por lo que **\*** y **/** se evalúan antes que **+** y **-**. La recursión por la izquierda ( $e \rightarrow e + t$ ) hace que las operaciones del mismo nivel se agrupen de izquierda a derecha.

```
e -> e + t | e - t | t
t -> t * f | t / f | f
f -> ( e ) | num
```



```
GNU nano 7.2 G1.g4 *
# Gramática G1 - Asociatividad izquierda, precedencia correcta (* / > + -)
grammar G1;

prog : e EOF ;

e : e '+' t # G1Suma
  | e '-' t # G1Resta
  | t      # G1TermE
  ;

t : t '*' f # G1Mult
  | t '/' f # G1Div
  | f      # G1TermT
  ;

f : '(' e ')' # G1Paren
  | NUM      # G1Num
  ;

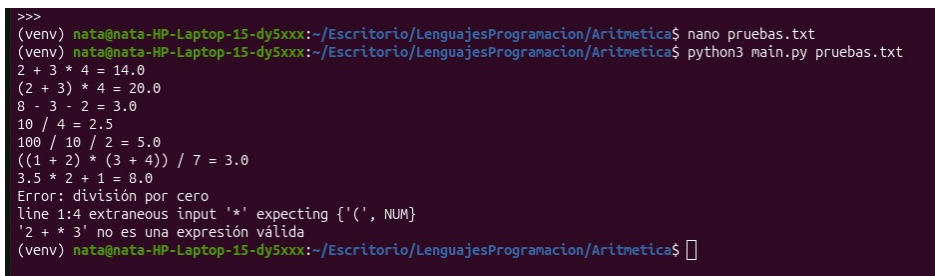
NUM : [0-9]+ ('.' [0-9]+)? ;
WS  : [ \t\r\n]+ -> skip ;
```

Figura 2: Gramática G1 con asociatividad a la izquierda

A partir del archivo **G1.g4**, ANTLR generó el lexer, el parser y la clase base del Visitor:

```
antlr4 -Dlanguage=Python3 -visitor -no-listener G1.g4
```

Luego se creó **EvalVisitor.py**, con un método por cada etiqueta de la gramática (por ejemplo, **visitG1Suma** suma el valor de sus dos hijos), y **main.py**, que une las tres fases: análisis léxico, análisis sintáctico y evaluación. El programa lee un archivo de texto y evalúa cada línea, reportando las expresiones inválidas y la división por cero.



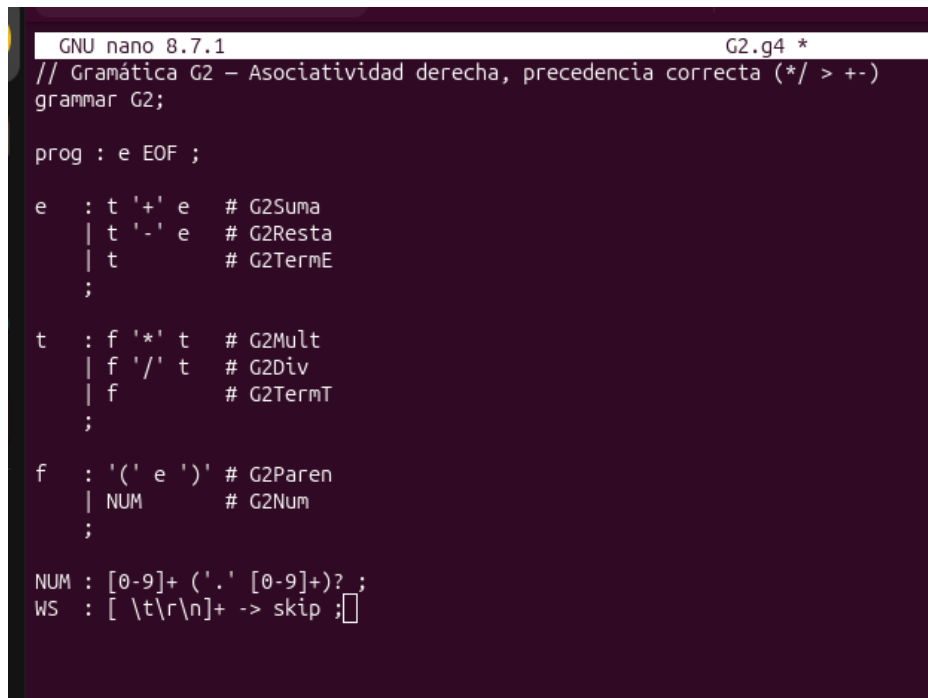
```
>>>
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano pruebas.txt
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ python3 main.py pruebas.txt
2 + 3 * 4 = 14.0
(2 + 3) * 4 = 20.0
8 - 3 - 2 = 3.0
10 / 4 = 2.5
100 / 10 / 2 = 5.0
((1 + 2) * (3 + 4)) / 7 = 3.0
3.5 * 2 + 1 = 8.0
Error: división por cero
line 1:4 extraneous input '*' expecting {'(', NUM}
'2 + * 3' no es una expresión válida
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$
```

Figura 3: Pruebas de la gramática G1

## 4. Gramática G2: asociatividad derecha

Para obtener asociatividad por la derecha se invirtió la recursión: en lugar de  $e \rightarrow e + t$  se escribe  $e \rightarrow t + e$ . La precedencia se conserva porque se mantienen los tres niveles.

```
e -> t + e | t - e | t
t -> f * t | f / t | f
f -> ( e ) | num
```



```
GNU nano 8.7.1 G2.g4 *
// Gramática G2 - Asociatividad derecha, precedencia correcta (* / > + -)
grammar G2;

prog : e EOF ;

e  : t '+' e # G2Suma
   | t '-' e # G2Resta
   | t      # G2TermE
   ;

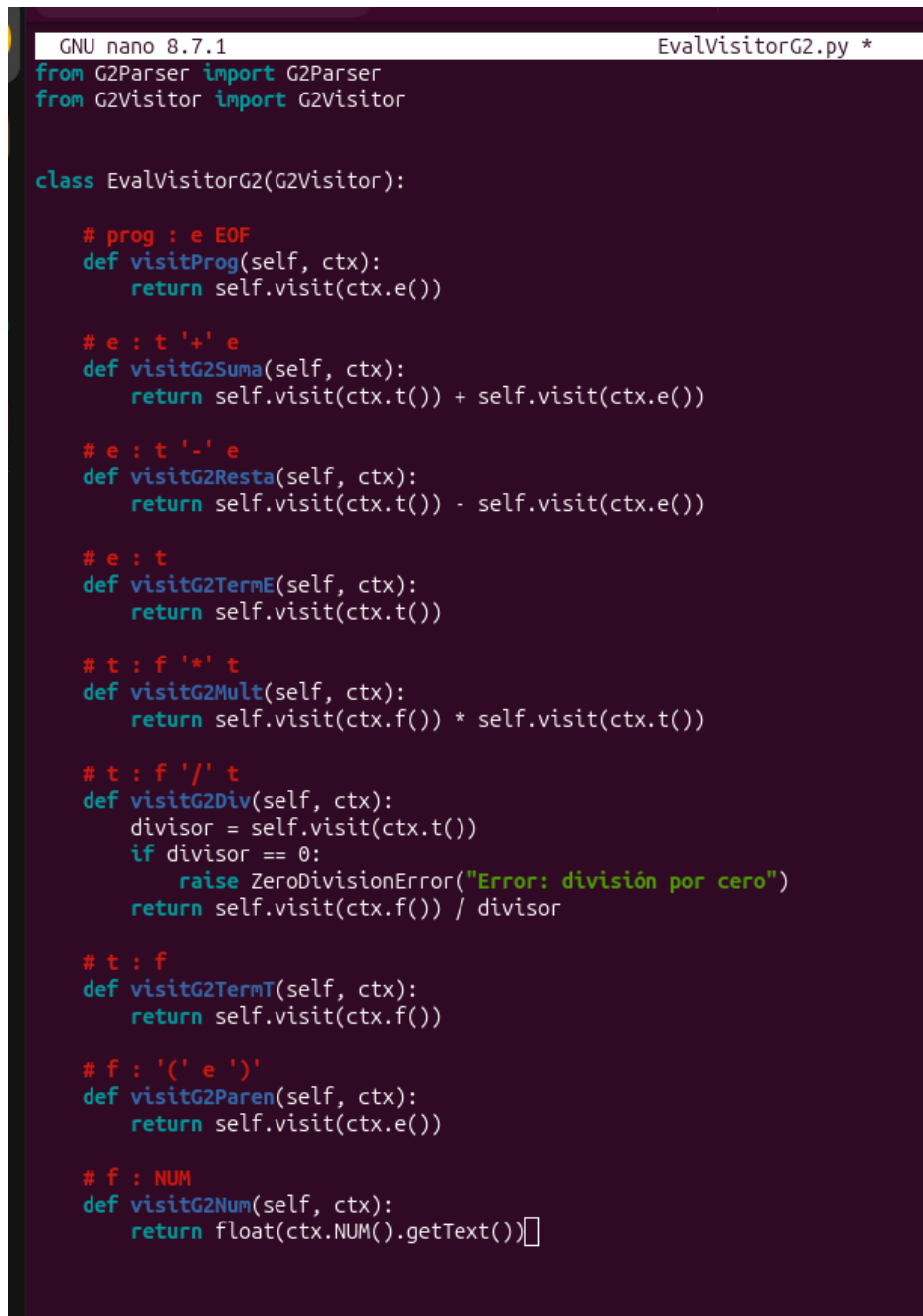
t  : f '*' t # G2Mult
   | f '/' t # G2Div
   | f      # G2TermT
   ;

f  : '(' e ')' # G2Paren
   | NUM      # G2Num
   ;

NUM : [0-9]+ ('.' [0-9]+)? ;
WS  : [ \t\r\n]+ -> skip ;
```

Figura 4: Gramática G2 con asociatividad a la derecha

Se creó un evaluador propio, `EvalVisitorG2.py`, en el que el operando izquierdo es `t` (o `f`) y el derecho es `e` (o `t`), al contrario que en `G1`.



```

GNU nano 8.7.1                               EvalVisitorG2.py *
from G2Parser import G2Parser
from G2Visitor import G2Visitor

class EvalVisitorG2(G2Visitor):

    # prog : e EOF
    def visitProg(self, ctx):
        return self.visit(ctx.e())

    # e : t '+' e
    def visitG2Suma(self, ctx):
        return self.visit(ctx.t()) + self.visit(ctx.e())

    # e : t '-' e
    def visitG2Resta(self, ctx):
        return self.visit(ctx.t()) - self.visit(ctx.e())

    # e : t
    def visitG2TermE(self, ctx):
        return self.visit(ctx.t())

    # t : f '*' t
    def visitG2Mult(self, ctx):
        return self.visit(ctx.f()) * self.visit(ctx.t())

    # t : f '/' t
    def visitG2Div(self, ctx):
        divisor = self.visit(ctx.t())
        if divisor == 0:
            raise ZeroDivisionError("Error: división por cero")
        return self.visit(ctx.f()) / divisor

    # t : f
    def visitG2TermT(self, ctx):
        return self.visit(ctx.f())

    # f : '(' e ')'
    def visitG2Paren(self, ctx):
        return self.visit(ctx.e())

    # f : NUM
    def visitG2Num(self, ctx):
        return float(ctx.NUM().getText())

```

Figura 5: EvalVisitor para G2

El programa principal mainG2.py se obtuvo copiando main.py y reemplazando las referencias de G1 por G2:

```
sed 's/G1/G2/g; s/EvalVisitor/EvalVisitorG2/g' main.py > mainG2.py
```

```
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ sed 's/G1/G2/g; s/EvalVisitor/
EvalVisitorG2/g' main.py > mainG2.py
head -n 5 mainG2.py
import sys
from antlr4 import InputStream, CommonTokenStream
from G2Lexer import G2Lexer
from G2Parser import G2Parser
from EvalVisitorG2 import EvalVisitorG2
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ cat pruebas.txt
```

Figura 6: Creación del main para G2

```
from EvalVisitorG2 import EvalVisitorG2
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ python3 mainG2.py pruebas.txt
2 + 3 * 4 = 14.0
(2 + 3) * 4 = 20.0
8 - 3 - 2 = 7.0
10 / 4 = 2.5
100 / 10 / 2 = 20.0
((1 + 2) * (3 + 4)) / 7 = 3.0
3.5 * 2 + 1 = 8.0
Error: división por cero
line 1:4 extraneous input '*' expecting '{', NUM}
'2 + * 3' no es una expresión válida
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$
```

Figura 7: Pruebas de G2

La diferencia aparece en las operaciones no asociativas (resta y división). La suma y la multiplicación dan igual en ambas gramáticas.

| Expresión    | G1 (izquierda)    | G2 (derecha)      |
|--------------|-------------------|-------------------|
| 8 - 3 - 2    | $(8 - 3) - 2 = 3$ | $8 - (3 - 2) = 7$ |
| 100 / 10 / 2 | $(100/10)/2 = 5$  | $100/(10/2) = 20$ |
| 2 + 3 * 4    | 14                | 14                |

Tabla 2: Comparación de resultados entre G1 y G2

## 5. Gramática G3: precedencia invertida

En G3 se conservó la asociatividad por la izquierda, pero se intercambiaron los operadores entre los niveles *e* y *t*. Así, la suma y la resta quedan en el nivel más profundo y se evalúan antes que la multiplicación y la división.

```
e -> e * t | e / t | t
t -> t + f | t - f | f
f -> ( e ) | num
```

```

GNU nano 8.7.1 G3.g4 *
// Gramática G3 - Asociatividad izquierda, precedencia invertida (+ - > */)
grammar G3;

prog : e EOF ;

e : e '*' t # G3Mult
  | e '/' t # G3Div
  | t # G3TermE
  ;

t : t '+' f # G3Suma
  | t '-' f # G3Resta
  | f # G3TermT
  ;

f : '(' e ')' # G3Paren
  | NUM # G3Num
  ;

NUM : [0-9]+ ('.' [0-9]+)? ;
WS : [ \t\r\n]+ -> skip ;

```

Figura 8: G3: asociatividad a la izquierda y precedencia invertida

Se crearon EvalVisitorG3.py y mainG3.py siguiendo el mismo procedimiento de G2.

```

'2 + * 3' no es una expresión válida
(nenv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano G3.g4
(nenv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ antlr4 -Dlanguage=Python3 -vis
itor -no-listener G3.g4
ls
EvalVisitor.py  G1Lexer.interp  G2.g4          G2Lexer.tokens  G3.tokens      G3Visitor.py  venv
EvalVisitorG2.py G1Lexer.py      G2.interp      G2Parser.py     G3Lexer.interp __pycache__
G1.g4          G1Lexer.tokens  G2.tokens      G2Visitor.py    G3Lexer.py     main.py
G1.interp      G1Parser.py     G2Lexer.interp G3.g4           G3Lexer.tokens mainG2.py
G1.tokens      G1Visitor.py    G2Lexer.py     G3.interp       G3Parser.py    pruebas.txt
(nenv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano EvalVisitorG3.py
(nenv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ sed 's/G1/G3/g; s/EvalVisitor/
EvalVisitorG3/g' main.py > mainG3.py
head -n 5 mainG3.py
import sys
from antlr4 import InputStream, CommonTokenStream
from G3Lexer import G3Lexer
from G3Parser import G3Parser
from EvalVisitorG3 import EvalVisitorG3
(nenv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$

```

Figura 9: Main y EvalVisitor para G3

```

from EvalvstG3 import EvalvstG3
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ echo "==== G1 (precedencia normal) ====="
python3 main.py pruebas.txt
echo "==== G3 (precedencia invertida) ====="
python3 mainG3.py pruebas.txt
==== G1 (precedencia normal) =====
2 + 3 * 4 = 14.0
(2 + 3) * 4 = 20.0
8 - 3 - 2 = 3.0
10 / 4 = 2.5
100 / 10 / 2 = 5.0
((1 + 2) * (3 + 4)) / 7 = 3.0
3.5 * 2 + 1 = 8.0
Error: división por cero
line 1:4 extraneous input '*' expecting {'(', NUM}
'2 + * 3' no es una expresión válida
==== G3 (precedencia invertida) =====
2 + 3 * 4 = 20.0
(2 + 3) * 4 = 20.0
8 - 3 - 2 = 3.0
10 / 4 = 2.5
100 / 10 / 2 = 5.0
((1 + 2) * (3 + 4)) / 7 = 3.0
3.5 * 2 + 1 = 10.5
Error: división por cero
line 1:4 extraneous input '*' expecting {'(', NUM}
'2 + * 3' no es una expresión válida
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$

```

Figura 10: Pruebas de G3

Solo cambian las expresiones que mezclan operadores de distinto nivel. Las que tienen paréntesis o un solo tipo de operador dan el mismo resultado que en G1.

| Expresión     | G1                       | G3                          |
|---------------|--------------------------|-----------------------------|
| $2 + 3 * 4$   | $2 + (3 \times 4) = 14$  | $(2 + 3) \times 4 = 20$     |
| $3.5 * 2 + 1$ | $(3.5 \times 2) + 1 = 8$ | $3.5 \times (2 + 1) = 10.5$ |
| $(2 + 3) * 4$ | 20                       | 20                          |
| $8 - 3 - 2$   | 3                        | 3                           |

Tabla 3: Comparación de resultados entre G1 y G3

## 6. Gramática G4: un nivel por operador

En G4 cada operador tiene su propio nivel de precedencia, con el orden división > multiplicación > resta > suma y asociatividad por la izquierda. Por claridad, las reglas se nombraron según el operador que manejan.

```

suma -> suma + resta | resta
resta -> resta - mult | mult
mult -> mult * div | div
div -> div / factor | factor
factor -> ( suma ) | num

```

```

GNU nano 8.7.1                                G4.g4 *
// Gramática G4 – Precedencia: / > * > - > +, asociatividad izquierda
grammar G4;

prog  : suma EOF ;

suma  : suma '+' resta      # G4Suma
      | resta              # G4PasaResta
      ;

resta  : resta '-' mult     # G4Resta
      | mult              # G4PasaMult
      ;

mult   : mult '*' div      # G4Mult
      | div              # G4PasaDiv
      ;

div    : div '/' factor    # G4Div
      | factor          # G4PasaFactor
      ;

factor : '(' suma ')'      # G4Paren
      | NUM              # G4Num
      ;

NUM    : [0-9]+ ('.' [0-9]+)? ;
WS     : [ \t\r\n]+ -> skip ;

```

Figura 11: G4: un nivel de precedencia por operador

```

'2 * 3' no es una expresión válida
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano G4.g4
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ antlr4 -Dlanguage=Python3 -vis
itor -no-listener G4.g4
ls
EvalVisitor.py  G1Lexer.interp  G2.interp      G2Visitor.py  G3Lexer.tokens  G4Lexer.interp  main.py
EvalVisitorG2.py G1Lexer.py      G2.tokens      G3.g4         G3Parser.py     G4Lexer.py      mainG2.py
EvalVisitorG3.py G1Lexer.tokens  G2Lexer.interp G3.interp     G3Visitor.py    G4Lexer.tokens  mainG3.py
G1.g4          G1Parser.py     G2Lexer.py     G3.tokens     G4.g4          G4Parser.py     pruebas.txt
G1.interp      G1Visitor.py    G2Lexer.tokens G3Lexer.interp G4.interp      G4Visitor.py    venv
G1.tokens      G2.g4          G2Parser.py    G3Lexer.py    G4.tokens      __pycache__
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano EvalVisitorG4.py
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ sed 's/G1/G4/g; s/EvalVisitor/
EvalVisitorG4/g' main.py > mainG4.py
head -n 5 mainG4.py
import sys
from antlr4 import InputStream, CommonTokenStream
from G4Lexer import G4Lexer
from G4Parser import G4Parser
from EvalVisitorG4 import EvalVisitorG4
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$

```

Figura 12: EvalVisitor y main para G4

Para esta gramática se creó un archivo de pruebas propio, `pruebas_g4.txt`, con expresiones que combinan multiplicación con división y suma con resta.

```

GNU nano 8.7.1                                pruebas_g4.txt *
2 + 3 * 4
10 + 2 - 3
10 - 2 + 3
2 * 8 / 4
12 / 2 * 3
(10 + 2) * 3
0.1 * 3 / 3
0.1 + 0.2 - 0.3

```

Figura 13: Archivo de pruebas para G4

```

from EvalvsttorG4 import EvalvsttorG4
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ nano pruebas_g4.txt
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ echo "==== G1 ====="
python3 main.py pruebas_g4.txt
echo "==== G4 ====="
python3 mainG4.py pruebas_g4.txt
==== G1 =====
2 + 3 * 4 = 14.0
10 + 2 - 3 = 9.0
10 - 2 + 3 = 11.0
2 * 8 / 4 = 4.0
12 / 2 * 3 = 18.0
(10 + 2) * 3 = 36.0
0.1 * 3 / 3 = 0.100000000000000002
0.1 + 0.2 - 0.3 = 5.551115123125783e-17
==== G4 =====
2 + 3 * 4 = 14.0
10 + 2 - 3 = 9.0
10 - 2 + 3 = 11.0
2 * 8 / 4 = 4.0
12 / 2 * 3 = 18.0
(10 + 2) * 3 = 36.0
0.1 * 3 / 3 = 0.1
0.1 + 0.2 - 0.3 = 2.7755575615628914e-17
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ 

```

Figura 14: Pruebas de G4

En aritmética exacta los resultados coinciden con G1, ya que  $a \times b/c = a \times (b/c)$  y  $a + b - c = a + (b - c)$ . Sin embargo, el árbol es distinto, y la diferencia se hace visible con números decimales por los errores de redondeo del punto flotante:

| Expresión         | G1                     | G4                     |
|-------------------|------------------------|------------------------|
| $2 * 8 / 4$       | $(2 \times 8)/4 = 4$   | $2 \times (8/4) = 4$   |
| $0.1 * 3 / 3$     | 0.100000000000000002   | 0.1                    |
| $0.1 + 0.2 - 0.3$ | $5.55 \times 10^{-17}$ | $2.78 \times 10^{-17}$ |

Tabla 4: Comparación de resultados entre G1 y G4

## 7. Generación de los AST

Para visualizar la estructura de cada expresión se instaló Graphviz, la herramienta que dibuja los árboles.

```
sudo apt install graphviz -y
```

```
(venv) nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/Aritmetica$ sudo apt install graphviz -y
dot -V
[sudo: authenticate] Contraseña:
Instalando:
  graphviz

Instalando dependencias:
  libcdt6 libcgraph8 libgvc7 libgvplugin-gd8 libgvplugin-pango8 libgvpr2 libpathplan4 libxdot4

Paquetes sugeridos:
  gsfonts graphviz-doc

Resumen:
  Actualizando: 0, Instalando 9, Eliminando: 0, no actualizando: 13
  Tamaño de la descarga: 760 kB
  Espacio necesario: 2.239 kB / 71,3 GB disponible

Des:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libcdt6 amd64 14.1.2-1ubuntu1 [20,1 kB]
Des:2 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libcgraph8 amd64 14.1.2-1ubuntu1 [55,3 kB]
Des:3 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libpathplan4 amd64 14.1.2-1ubuntu1 [26,3 kB]
Des:4 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libxdot4 amd64 14.1.2-1ubuntu1 [23,5 kB]
Des:5 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libgvc7 amd64 14.1.2-1ubuntu1 [397 kB]
Des:6 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libgvpr2 amd64 14.1.2-1ubuntu1 [127 kB]
Des:7 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libgvplugin-gd8 amd64 14.1.2-1ubuntu1 [26,4 kB]
Des:8 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libgvplugin-pango8 amd64 14.1.2-1ubuntu1 [24,6 kB]
Des:9 http://archive.ubuntu.com/ubuntu resolute/universe amd64 graphviz amd64 14.1.2-1ubuntu1 [58,5 kB]
Descargados 760 kB en 3s (257 kB/s)
Seleccionando el paquete libcdt6:amd64 previamente no seleccionado.
(Leyendo la base de datos ... 210154 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../0-libcdt6_14.1.2-1ubuntu1_amd64.deb ...
Desempaquetando libcdt6:amd64 (14.1.2-1ubuntu1) ...
Seleccionando el paquete libcgraph8:amd64 previamente no seleccionado.
Preparando para desempaquetar .../1-libcgraph8_14.1.2-1ubuntu1_amd64.deb ...
Desempaquetando libcgraph8:amd64 (14.1.2-1ubuntu1) ...
Seleccionando el paquete libpathplan4:amd64 previamente no seleccionado.
Preparando para desempaquetar .../2-libpathplan4_14.1.2-1ubuntu1_amd64.deb ...
```

Figura 15: Instalación de Graphviz

Se escribió el script `graficar_ast.py`, que funciona con las cuatro gramáticas. Para cada expresión del archivo de pruebas: construye el AST a partir del árbol de ANTLR (conservando solo operadores y números), lo evalúa, lo muestra en texto con paréntesis explícitos y lo guarda como imagen PNG en una carpeta por gramática (`ast_G1`, `ast_G2`, etc.).

```

GNU nano 8.7.1                                graficar_ast.py *
import sys
import os
import importlib
import subprocess
from antlr4 import InputStream, CommonTokenStream, Token
from antlr4.tree.Tree import TerminalNode

class Nodo:
    """Nodo del AST: un operador con dos hijos, o un número sin hijos."""
    def __init__(self, valor, hijos=None):
        self.valor = valor
        self.hijos = hijos or []

def construir_ast(arbol):
    """Construye el árbol sintáctico de ANTLR en un AST."""
    if isinstance(arbol, TerminalNode):
        return Nodo(arbol.getText())

    hijos = [arbol.getChild(i) for i in range(arbol.getChildCount())]
    hijos = [h for h in hijos
              if not (isinstance(h, TerminalNode) and h.getSymbol().type == Token.EOF)]

    # Nodo que solo pasa a otro nivel (e -> t, t -> f): se salta
    if len(hijos) == 1:
        return construir_ast(hijos[0])

    # Paréntesis: se queda solo con la de dentro
    if len(hijos) == 3 and isinstance(hijos[0], TerminalNode) and hijos[0].getText() == "(":
        return construir_ast(hijos[1])

    # Operación binaria: izquierda OPERADOR derecha
    if len(hijos) == 3:
        operador = hijos[1].getText()
        return Nodo(operador, [construir_ast(hijos[0]), construir_ast(hijos[2])])

    raise ValueError("Estructura no reconocida")

```

Figura 16: Script para graficar los AST

```

for g in G1 G2 G3; do python3 graficar_ast.py $g pruebas.txt; done
python3 graficar_ast.py G4 pruebas_g4.txt

```

```

(venv) natagnata-HP-Laptop-15-dy5xxx1~/Escritorio/LenguajesProgramacion/Aritmetica$ python3 graficar_ast.py G1 pruebas.txt
===== G1 =====
[1] 2 + 3 * 4 -> AST: ((2 + (3 * 4))) = 14.0
[2] (2 + 3) * 4 -> AST: ((2 + 3) * 4) = 20.0
[3] 8 - 3 - 2 -> AST: ((8 - 3) - 2) = 3.0
[4] 10 / 4 -> AST: (10 / 4) = 2.5
[5] 100 / 10 / 2 -> AST: ((100 / 10) / 2) = 5.0
[6] ((1 + 2) * (3 + 4)) / 7 -> AST: (((1 + 2) * (3 + 4)) / 7) = 3.0
[7] 3.5 * 2 + 1 -> AST: ((3.5 * 2) + 1) = 8.0
[8] 5 / 0 -> AST: (5 / 0) = Error: división por cero
line 114 extraneous input '*' expecting '(', NUM
[9] 2 + * 3 -> expresión inválida, no se grafica
Imágenes guardadas en la carpeta ast_G1/

(venv) natagnata-HP-Laptop-15-dy5xxx1~/Escritorio/LenguajesProgramacion/Aritmetica$ for g in G2 G3; do python3 graficar_ast.py $g pruebas.txt; done
python3 graficar_ast.py G2 pruebas_g2.txt
===== G2 =====
[1] 2 + 3 * 4 -> AST: ((2 + (3 * 4))) = 14.0
[2] (2 + 3) * 4 -> AST: ((2 + 3) * 4) = 20.0
[3] 8 - 3 - 2 -> AST: ((8 - 3) - 2) = 3.0
[4] 10 / 4 -> AST: (10 / 4) = 2.5
[5] 100 / 10 / 2 -> AST: ((100 / 10) / 2) = 5.0
[6] ((1 + 2) * (3 + 4)) / 7 -> AST: (((1 + 2) * (3 + 4)) / 7) = 3.0
[7] 3.5 * 2 + 1 -> AST: ((3.5 * 2) + 1) = 8.0
[8] 5 / 0 -> AST: (5 / 0) = Error: división por cero
line 114 extraneous input '*' expecting '(', NUM
[9] 2 + * 3 -> expresión inválida, no se grafica
Imágenes guardadas en la carpeta ast_G2/

(venv) natagnata-HP-Laptop-15-dy5xxx1~/Escritorio/LenguajesProgramacion/Aritmetica$ for g in G3; do python3 graficar_ast.py $g pruebas.txt; done
python3 graficar_ast.py G3 pruebas_g3.txt
===== G3 =====
[1] 2 + 3 * 4 -> AST: ((2 + (3 * 4))) = 14.0
[2] (2 + 3) * 4 -> AST: ((2 + 3) * 4) = 20.0
[3] 8 - 3 - 2 -> AST: ((8 - 3) - 2) = 3.0
[4] 10 / 4 -> AST: (10 / 4) = 2.5
[5] 100 / 10 / 2 -> AST: ((100 / 10) / 2) = 5.0
[6] ((1 + 2) * (3 + 4)) / 7 -> AST: (((1 + 2) * (3 + 4)) / 7) = 3.0
[7] 3.5 * 2 + 1 -> AST: ((3.5 * 2) + 1) = 8.0
[8] 5 / 0 -> AST: (5 / 0) = Error: división por cero
line 114 extraneous input '*' expecting '(', NUM
[9] 2 + * 3 -> expresión inválida, no se grafica
Imágenes guardadas en la carpeta ast_G3/

(venv) natagnata-HP-Laptop-15-dy5xxx1~/Escritorio/LenguajesProgramacion/Aritmetica$ for g in G4; do python3 graficar_ast.py $g pruebas_g4.txt; done
python3 graficar_ast.py G4 pruebas_g4.txt
===== G4 =====
[1] 2 + 3 * 4 -> AST: ((2 + (3 * 4))) = 14.0
[2] 10 + 2 - 3 -> AST: (10 + (2 - 3)) = 9.0
[3] 10 - 2 + 3 -> AST: ((10 - 2) + 3) = 11.0
[4] 2 * 8 / 4 -> AST: (2 * (8 / 4)) = 4.0
[5] 12 / 2 * 3 -> AST: ((12 / 2) * 3) = 18.0
[6] (10 + 2) * 3 -> AST: ((10 + 2) * 3) = 36.0
[7] 0.1 * 3 / 3 -> AST: (0.1 * (3 / 3)) = 0.1
[8] 0.1 + 0.2 - 0.3 -> AST: (0.1 + (0.2 - 0.3)) = 2.775575615628914e-17
Imágenes guardadas en la carpeta ast_G4/
(venv) natagnata-HP-Laptop-15-dy5xxx1~/Escritorio/LenguajesProgramacion/Aritmetica$

```

Figura 17: Generación de los gráficos de todas las gramáticas

El árbol sintáctico que genera ANTLR incluye todos los nodos de la gramática (e, t, f), los paréntesis y los pasos intermedios. El AST es una versión simplificada que solo conserva operadores y números: los paréntesis desaparecen porque la agrupación ya queda representada en la forma del árbol.

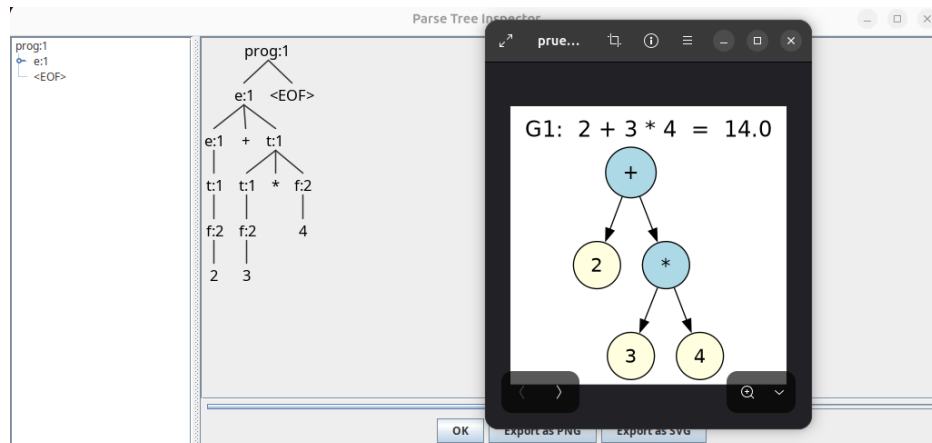


Figura 18: Comparación entre el árbol sintáctico y el AST generado

## 8. Análisis de resultados

### 8.1. Asociatividad: G1 y G2

Con la expresión  $8 - 3 - 2$ , el AST de G1 crece hacia la izquierda y el de G2 hacia la derecha. Por eso G1 calcula  $(8 - 3) - 2 = 3$  y G2 calcula  $8 - (3 - 2) = 7$ .

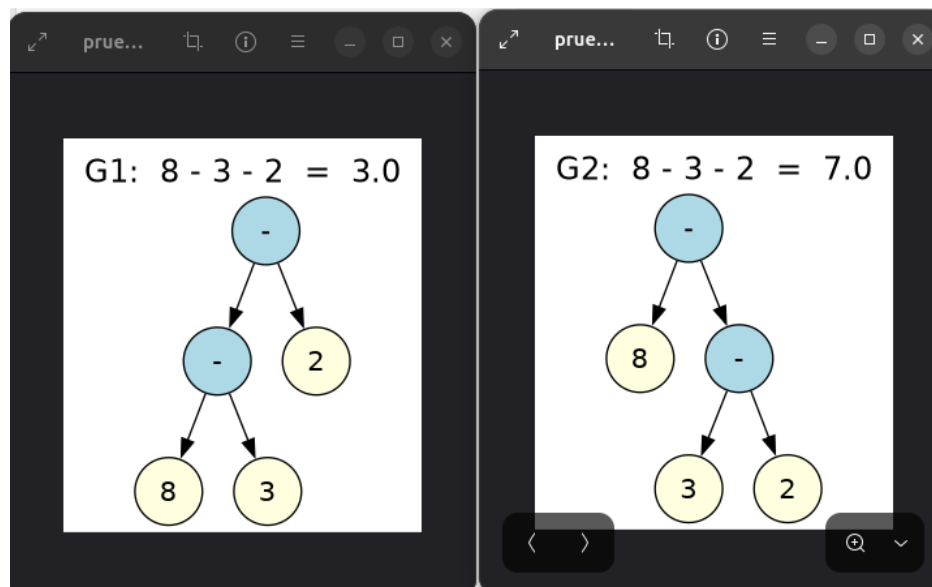


Figura 19: Diferencia de asociatividad entre G1 y G2 (ejemplo 3)

### 8.2. Precedencia: G1 y G3

Con  $2 + 3 * 4$ , en G1 la raíz es el  $+$  y la multiplicación queda abajo, por lo que se evalúa primero (resultado 14). En G3 la raíz es el  $*$  y la suma se evalúa primero (resultado 20). El operador que queda más abajo en el árbol es el que se ejecuta primero.

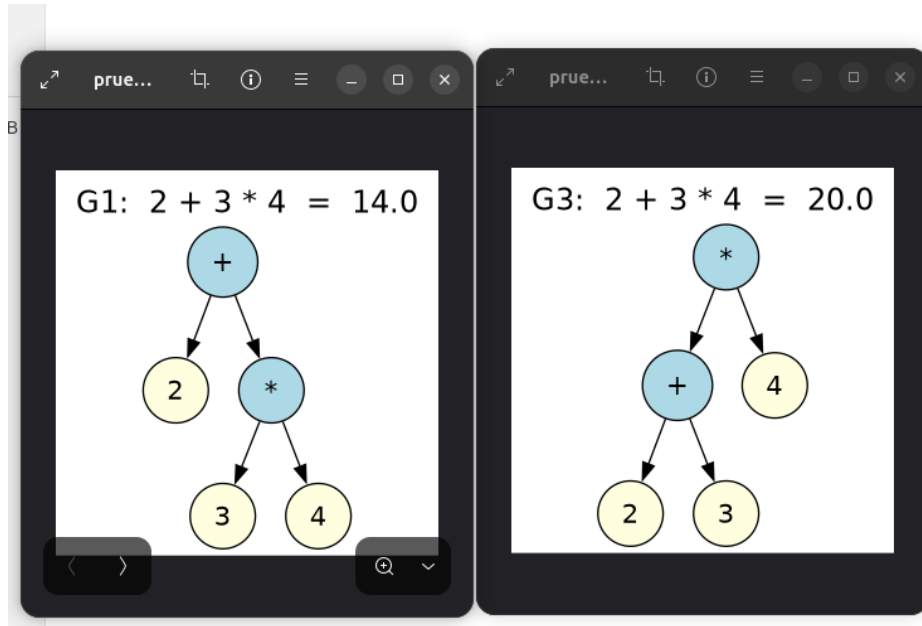


Figura 20: Diferencia de precedencia entre G1 y G3 (ejemplo 1)

### 8.3. Precedencia en G4

En G4 la división queda por debajo de la multiplicación y la resta por debajo de la suma. Los árboles son distintos a los de G1 aunque los resultados coincidan en aritmética exacta.

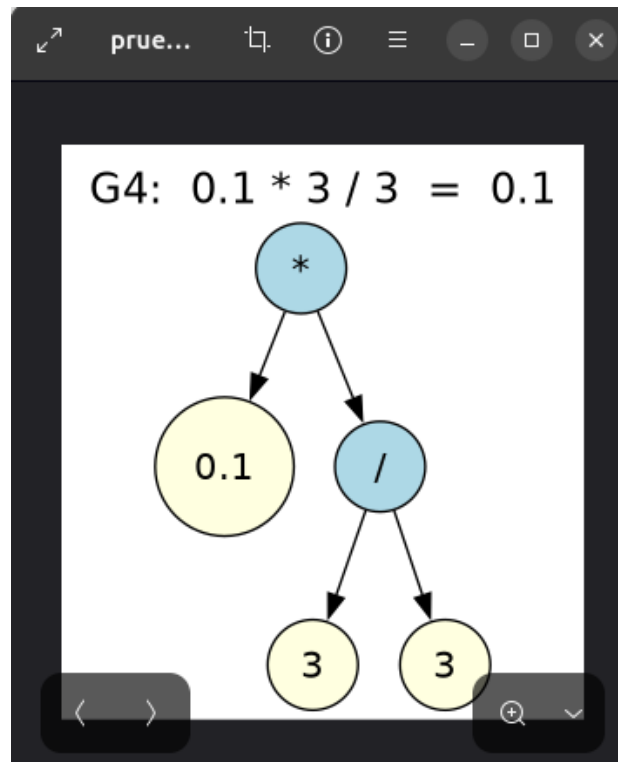


Figura 21: Precedencia en G4

## 9. Conclusiones

- La **precedencia** se define por el nivel de la gramática en que aparece cada operador: cuanto más profundo queda en el árbol, antes se evalúa.
- La **asociatividad** se define por el lado de la recursión: la recursión por la izquierda agrupa de izquierda a derecha, y la recursión por la derecha, al revés.
- La asociatividad solo cambia el resultado en operaciones no asociativas, como la resta y la división.
- Dos gramáticas pueden reconocer exactamente las mismas expresiones y, aun así, asignarles árboles y resultados diferentes.
- Separar el análisis sintáctico (parser) de la evaluación (Visitor) permite cambiar el significado de un lenguaje sin modificar su gramática, y reutilizar la misma estructura para distintos fines, como evaluar o graficar el AST.